

lec 115

Q.) Maximal Square

→ find largest square formed in a matrix.

recursion $i \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} j$

```
int solve (vector<vector> matrix,
int i, int j, int & maxi) {
if (i >= mat.size() || j >=
mat[0].size())
return 0;
```

```
int right = solve (matrix, i,
j+1, maxi);
```

```
int diagonal = solve (matrix,
i+1, j+1, maxi);
```

```
int down = solve (matrix, i+1,
j, maxi);
```

```
if (mat[i][j] == 1) {
```

```
int ans = 1 + min (right,
min (diagonal, down);
```

```
maxi = max (ans, maxi);
return ans;
```

```
} else
```

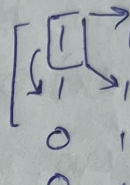
```
{
return 0;
```

```
int maxSquare (int n, int m, matrix) {
int maxi = 0;
solve (mat, 0, 0, maxi);
return maxi;
```

```
}
```

approach : Extend square

from right down & diagonal.



Tabulation

```
int solveTab (vector<vector<int>>
& mat, int & maxi) {
```

```
int row = mat.size();
```

```
int col = mat[0].size();
```

```
vector<vector<int>> dp (row+1,
```

```
vector<int> (col+1, 0));
```

```
for (int i = row-1; i >= 0; i--) {
```

```
for (int j = col-1; j >= 0; j--) {
```

```
int right = dp[i][j+1];
```

```
int diagonal = dp[i+1][j+1];
```

```
int down = dp[i+1][j];
```

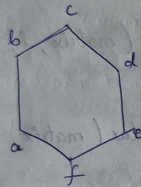
FF = 1000

if (mat[i][j] == 1) {
 $dp[i][j] = 1 + \min(\text{right},$
 $\min(\text{right}, \min(\text{diagonal}, \text{down}));$
 $\text{maxi} = \max(\text{maxi}, dp[i][j]);$
 }
 else
 {
 $dp[i][j] = 0;$
 }
 }
 return dp[0][0];

polygon with 5 sides



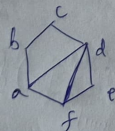
Note: to divide any polygon
 of sides $\rightarrow n$ into triangles
 you have to draw $(n-3)$
 diagonals.
 & and polygon has total 'n'
 diagonals



vertices = {a, b, c, d, e, f}

$\rightarrow$ we are sure that a & f
 are adjacent vertices.

$\rightarrow$ so keeping them as a
 base of Δ , we draw
 a triangle by selecting
 any vertex.



$ans = adf + \text{solve}(a, d)$
 $+ \text{solve}(d, f)$

recursion relation becomes.

$\text{solve}(i, j) = (adf) + \text{solve}(i, k)$
 $+ \text{solve}(k, j).$

base case

if ($i+1 == j$)
 return 0;

Recursion

$\text{int solve}(\text{vector } v, \text{int } i, \text{int } j)$

// base

if ($i+1 == j$) {
 return 0;
}

$\text{int ans} = \text{INT_MAX};$

for ($\text{int } k = i+1; k < j; k++$) {

$\text{ans} = \min(\text{ans},$

$v[i]^*v[j]^*v[k] + \text{solve}$
 $(v, i, k) + \text{solve}(v, k, j));$

}

return ans;

}

$\text{int minScoreTriangular}(\text{vector } <\text{int}> \& \text{values})$

{

$\text{int } n = \text{values.size}();$

return $\text{solve}(\text{values},$

$\text{values[0]}, n-1);$

}

Recursion + mem

$\text{int solveMem}(\text{vector } v, \text{int } i, \text{int } j,$
 $\text{vector } <\text{vector}> \& dp)$ {

// base case

if ($i+1 == j$) return 0;

if ($dp[i][j] != -1$)
 return $dp[i][j];$

$\text{int ans} = \text{INT_MAX};$

for ($\text{int } k = i+1; k < j; k++$) {

$\text{ans} = \min(\text{ans}, v[i]^*v[j]^*v[k]$

$+ \text{solveMem}(v, i, k, dp) +$

$\text{solveMem}(v, k, j, dp));$

}

$dp[i][j] = \text{ans};$

return $dp[i][j];$

}

$\text{int minScoreTriangular}(\text{vector } <\text{int}> \& \text{values})$

{

$\text{int } n = \text{values.size}();$

$\text{vector } <\text{vector } <\text{int}>> dp(n,$

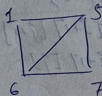
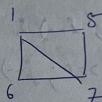
$\text{vector}(n, -1));$

return $\text{solveMem}(\text{values}, 0, n-1,$

$dp);$

}

Q. min score triangulation of
 polygon



final
 ans = 77

240

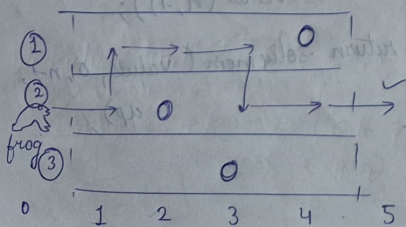
Tabulation

```
int solveTab(vector<int> v) {
    int n = v.size();
    vector<vector<int>> dp(n,
        vector<int>(n, 0));

    for (int i = n-1; i >= 0; i--) {
        for (int j = i+2; j < n; j++) {
            int ans = INT_MAX;
            for (int k = i+1; k < j; k++) {
                ans = min(ans, v[i] * v[j] * v[k]
                    + dp[i][k] + dp[k][j]);
            }
            dp[i][j] = ans;
        }
    }
    return dp[0][n-1];
}
```

lec 17

Q) Minimum Sideways Jump



target get frog from '0' index to '5' index
you are given an obstacle array which tells you lane no. at which obstacle is present at each index of array.

eg
 0 1 2 3 4
 obs = [0, 2, 3, 1, 0]
 ↓
 means no obstacle
 sideways jumps

recursion + memoization

code

```
int solveMem(vector<int> &obstacles,
    int currlane, int currpos,
    vector<vector<int>> &dp) {
    // base case
    int n = obstacles.size() - 1;
    if (currpos == n) {
        return 0;
    }
    if (dp[currlane][currpos] != -1)
        return dp[currlane][currpos];
```

```
if (obstacles[currpos+1] !=
    currlane) {
    return solveMem(obstacles,
        currlane, currpos+1, dp);
}
else {
```

```
int ans = INT_MAX;
for (i = 1; i <= 3; i++) {
    if (currlane != i &&
        obstacles[currpos+1] != i)
```

```
ans = min(ans, 1 + solveMem(
    obstacles, i, currpos+1, dp));
}
```

```
dp[currlane][currpos] = ans;
return dp[currlane][currpos];
}
```

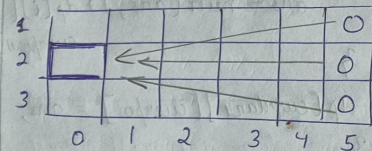
```
int minJumps(vector<int> &
    obstacles) {
```

```
vector<vector<int>> dp(4,
    vector<int>(obstacles.size(), -1));
return solve(obstacles, 2, 0, dp);
}
```

int solveTab (

Tabulation approach. (bottom up)

→ we start from (1, n) (2, n) & (3, n) cell and try to reach (2, 0) cell.



```
int solveTab(vector<int> &obstacles) {
    int n = obstacles.size() - 1;
    vector<vector<int>> dp(4,
        vector<int>(obstacles.size(), INT_MAX));

    dp[0][n] = 0;
    dp[1][n] = 0;
    dp[2][n] = 0;
    dp[3][n] = 0;
```

```
for (int currpos = n-1; currpos >= 0;
    currpos--) {
```

```
for (int currlane = 1; currlane <= 3;
    currlane++) {
```

```
if (obstacles[currpos+1] !=
    currlane) {
```

```
dp[currlane][currpos] =
    dp[currlane][currpos+1];
}
```


else {

```
int ans = 1e9;
for (int i=1; i<=3; i++){
    if (curvdane != i &&
        obstacles[curpos] != i)
```

```
ans = min(ans, 1 + dp[i][
    curpos+1]);
```

```
dp[curvdane][curpos] = ans;
```

```
} } for end #2
```

```
return dp[curvdane][curpos] = ans;
```

```
return min(dp[2][0],
```

```
min(dp[1][0]+1, dp[3][0]+1));
```

```
}
```

lec 116

Q. Reducing Dishes.

Satisfaction level = [-1, -8, 0, 5, -9]
for each dishes.

To prepare 1 dish → 1 unit of time.

A chef can discard any dish. (maybe all).

Sorting is allowed

This problem is similar to 0/1 knapsack.

↳ include & exclude tree.

recur + mem

code

```
int solve(vector satisfaction, int index,
    int time, vector <vector> &dp){
```

// base case

```
if (index == satisfaction.size())
```

```
return 0;
```

```
if (dp[index][time] != -1)
```

```
return dp[index][time];
```

```
int include = satisfaction[index]
    (time+1) + solve(satisfaction,
    index+1, time+1, dp);
```

```
int exclude = 0 + solve(satisfaction,
    index+1, time, dp);
```

```
return dp[index][time] = max(include, exclude);
```

```
max(include, exclude);
```

```
}
```

```
int maxSatisfaction(vector satisfaction){
```

```
sort(satisfaction.begin(),
    satisfaction.end());
```

```
int n = satisfaction.size();
```

```
vector <vector <int>> dp(n+1,
```

```
vector <int> (n+1, -1));
```

```
return solve(satisfaction, 0, 0,
    dp);
```

Tabulation

```
int solve(vector satisfaction){
```

```
int n = satisfaction.size();
```

```
vector <vector <int>> dp(n+1, vector<
```

```
int> (n+1, 0));
```

```
for (int index = n-1; index >= 0;
```

```
index--){
```

```
for (int index time = n+1; time
```

```
>= 0; time--){
```

```
int include = satisfaction[index]
```

```
* (time+1) + dp[index+1][time+1];
```

```
int exclude = 0 + dp[index+1][time]
```

```
dp[index][time] = max(include,
    exclude);
```

```
}
```

```
return dp[0][0];
```

```
}
```

lec 119

Q. Longest Increasing Subsequence (strictly increasing)

i/p → [10, 9, 2, 5, 3, 7, 101, 16]

o/p → 4

soln → [2, 5, 7, 101] ✓

[10 9 2 5]

include inc
with condn

excl.

↙ ↘ ↙ ↘

[]ⁱ []ⁱ []ⁱ []ⁱ

↓
base case →

code recur + mem

```
int solve(int n, arr[], int curr,
          int prev, vector<vector<int>>
          &dp) {
```

// base case

```
if (curr == n) {
```

```
    return 0;
}
```

```
if (dp[curr][prev+1] != -1)
    return dp[curr][prev+1];
```

// include

```
int take = 0;
```

```
if (prev == -1 || arr[curr] > arr[prev])
    take = 1 + solve(n, arr, curr+1,
                     curr, dp);
```

// exclude

```
int notTake = 0;
```

```
if (notTake = solve(n, arr,
                    curr+1, prev, dp);
```

```
return dp[curr][prev+1] =
    max(take, notTake);
```

```
}
```

int LIS (int n, int a[])

```
{
    vector<vector<int>> dp(
        n, vector<int>(n+1, -1));
    return solve(h, a, 0, -1, dp);
}
```

Tabulation

```
int solve (vector<arr> arr) {
    vector<vector> dp (n+1, vector(n+1, 0));
    for (int curr = n-1; curr >= 0; curr--) {
        for (int prev = curr-1; prev >= -1; prev--) {
            // include
            int take = 0;
            if (prev == -1 || arr[curr] > arr[prev])
                take = 1 + dp[curr+1][prev+1];
            // exclude
            int notTake = dp[curr+1][prev+1];
            dp[curr][prev+1] = max(take, notTake);
        }
    }
    return dp[0][0];
}
```

// exclude

```
int notTake = dp[curr+1][prev+1];
dp[curr][prev+1] = max(take, notTake);
return dp[0][0];
```

*** DP with Binary Search

i/p → [5, 8, 3, 7, 9, 1]
 ↑ ↑ ↑ ↑ ↑ ↑
 [5, 8, ~~3~~, ~~7~~, 9, ~~1~~] → 3
 [3, 7, 9, ~~1~~] → 3
 [1] → 1

approach

[5, 8, 3, 7, 9, 1]
 ↑ ↑ ↑ ↑ ↑ ↑
 [3, 7, 9] → [1, 7, 9]
 3 7 9
 1 is this LIS
 But its ← NO
 length is the ans

Two things

- 1) Agar current element jo insert karna hai, last element se bada ho → to insert at last
- 2) else, replace it with its next larger number.

[5, 8, 9] ⊕ it will replace '8'.

code

```
int solveOpt (int arr[]) {
    if (arr.size() == 0)
        return 0;
    vector<int> ans;
    ans.push_back(arr[0]);
    for (int i=1; i<n; i++) {
        if (arr[i] > ans.back()) {
            ans.push_back(arr[i]);
        } else {
            // find just bada element
            int index = lower_bound(
                ans.begin(), ans.end(), arr[i])
                - ans.begin();
            arr[index] = arr[i];
        }
    }
}
```

return ans.size();

TC → O(n log n)

SC → O(n)

Q. Russian doll envelopes

i/p $\rightarrow$ envelopes = $[[5, 4], [6, 4], [6, 7], [2, 3]]$

$\rightarrow$ You can put envelopes into each other if both height & width is small. (strictly).

o/p $\rightarrow [2, 3] \Rightarrow [5, 4] \Rightarrow [6, 7]$
 $\Rightarrow [3]$

approach

- $\rightarrow$ sort vector of envelopes width wise and if width is same sort it dec order of height
- $\rightarrow$ Then apply LIS of height height column $\rightarrow$ length of LIS is your ans

lec 20

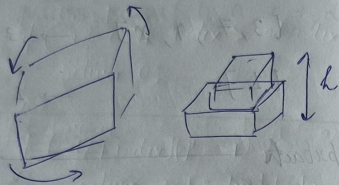
Q. maximum height by stacking cuboid

$\rightarrow$ n - cuboids are given.

cuboid $[i] = [l_i, b_i, h_i]$

you can place cuboid $[i]$ on $[j]$

if $\left\{ \begin{array}{l} l_i \leq l_j \\ b_i \leq b_j \\ h_i \leq h_j \end{array} \right\}$



return maximum height

NOTE

you can rotate cuboids

$l \leftrightarrow b \leftrightarrow h$

base = $(l \times b)$

Algo $\rightarrow$ i) sort each cuboids.

Such that it's height become maximum in all dimensions.

ii) Sort all cuboids in order of their increasing base. $(l \times b)$

e.g. $l \quad b \quad h$
i/p = $\begin{bmatrix} 50 & 45 & 20 \\ 95 & 37 & 53 \\ 45 & 23 & 12 \end{bmatrix}$

o/p $\rightarrow$ $\begin{bmatrix} 12 & 23 & 45 \\ 20 & 45 & 50 \\ 37 & 53 & 95 \end{bmatrix}$
transat $\rightarrow$ base $(l \times b)$ height

code (solving in form of LIS).
after sorting our input.

```
int maxHeight(vector<vector> &cuboids) {
    for (auto &a: cuboids) {
        sort(a.begin(), a.end());
    }
    sort(cuboids.begin(), cuboids.end());

    return value(cuboids.size(), cuboids);
}
```

```
bool check(vector<int> base, vector<int> newBox) {
    if (newBox[0] <= base[0] &&
        newBox[1] <= base[1] &&
        newBox[2] <= base[2])
        return true;
    else {
        return false;
    }
}
```

```
int solve(int n, vector<vector> &a) {
    vector<int> currRow(n+1, 0);
    vector<int> nextRow(n+1, 0);
```

```
for (int curr = n-1; curr >= 0; curr--) {
```

```
for (prev = n-1; prev >= 0; prev--) {
```

```
    // include
    int take = 0;
    if (prev == -1 || check(a[curr], a[prev])) {
        take = a[curr][2] + nextRow[prev+1];
    }
```

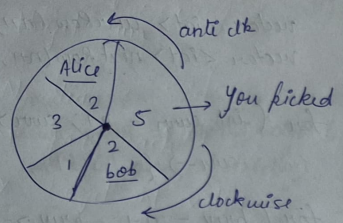
```
    // exclude
    int notTake = 0 + nextRow[prev+1];
```

```
    currRow[prev+1] = max(take, notTake);
    nextRow = currRow;
}
```

```
return nextRow[0];
}
```


Qc 121

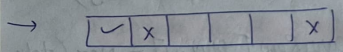
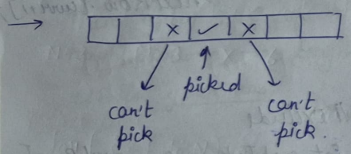
Q. Pizza with 3N slices.



return max no. of area u can pick.

approach :-

→ It is similar to house robbery question.



input : [1, 2, 3, 4, 5, 6]

dp : 10
↓
(6+4)

code :-
recursion + mem

```
int solveMem (int index, int endIndex,
vector<int> & slices, int n, vector<int> & dp) {
    if (n == 0 || index > endIndex) {
        return 0;
    }
```

```
    if (dp[index][n] != -1) {
        return dp[index][n];
    }
```

```
    int take = slices[index] + solveMem(
        index+2, endIndex, slices,
        n-1, dp);
```

```
    int notTake = 0 + solveMem(
        index+1, endIndex, slices, dp);
```

```
    return dp[index][n] = max(take,
        notTake);
}
```

```
int maxSizeSlices (vector<int> & slices) {
    int k = slices.size();
```

```
    vector<vector<int>> dp1 (k,
        vector<int> (k, -1));
```

```
    int case1 = solveMem (0,
        k-2, slices, k/3, dp1);
```

```
    vector<vector<int>> dp2 (k, vector<
        int> (k, -1));
```

```
    int case2 = solveMem (0, k-1, k/3,
        dp2);
```

```
    return max(case1, case2);
}
```

Tabulation.

```
int solveTab (vector<int> & slices) {
    int k = slices.size();
```

```
    vector<vector<int>> dp1 (k+2,
        vector<int> (k+2, 0));
```

```
    vector<vector<int>> dp (k+2,
        vector<int> (k+2, 0));
```

```
    for (int index = k-2, index >= 0;
        index--){
        for (int n = 1; n <= k/3; n++){
            int take = slices[index] +
                dp1[index+2][n-1];
```

```
            int notTake = 0 + dp1[index][n];
            dp1[index][n] = max(take,
                notTake);
        }
    }
```

```
    int case1 = dp1[0][k/3];
```

```
    for (int index = k-1, index >= 1;
        index--){
        for (int n = 1; n <= k/3; n++){
            int take = slices[index] +
                dp2[index+2][n-1];
```

```
            int notTake = 0 + dp2[index][n];
            dp2[index][n] = max(take,
                notTake);
        }
    }
```

```
    int case2 = dp2[0][k/3];
```

```
    return max(case1, case2);
}
```


Dec 22

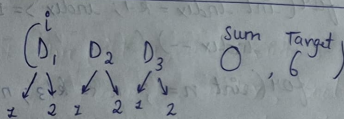
Q. Number of Dice rolls with target Sum.

'N' dices

'M' faces each

'X' target int value

return no. of ways you can make sum of all thrown N dices equal to 'X'



$N=3$ $M=3$ $X=6$

for ($i=1$ to N) {
for ($j=1$ to M) {

recursion

}

recursion mem

const int MOD = $1e9+7$;

int solve (int n, int k, int target,

vector <vector <int>> & dp) {

// base cases

if (target < 0) {

return 0;

}

if (target == 0 && n != 0) {

return 0;

}

if (target != 0 && n == 0) {

return 0;

}

if (target == 0 && n == 0) {

return 1;

}

if (dp[n][target] != -1) {

return dp[n][target];

}

long long ans = 0;

for (int i = 1; i <= k; i++) {

ans = (ans + solve (n-1, k, target-i))

% MOD;

return dp[n][target] = (int)ans;

}

int numRolls (int n, k, target) {

vector <vector <int>> dp (n,

vector <int> (target+1, -1);

return solve (n, k, target,

dp);

}

Tabulation

in Topdown $\rightarrow n \rightarrow n$ to 0

target = target to 0

bottomup = $n \rightarrow 0$ to n

target $\rightarrow 0$ to target.

code

int solveTab (int n, int k, int target) {

vector <vector <int>> dp (n,

vector <int> (target+1, 0);

dp[0][0] = 1;

for (int d = 1; d <= n; d++) {

for (int t = 1; t <= target; t++) {

long long ans = 0;

for (int i = 1; i <= min (k, t);

i++) {

ans = (ans + dp [d-1] [t-i]) % MOD;

}

dp [d] [t] = ans;

}

return dp [n] [target];

}

int numRolls (int n, k, target) {

vector <

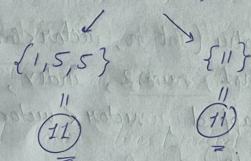
return solveTab (n, k, target);

}

Dec 23

Q. Partition Equal Subset Sum.

arr [] = {1, 5, 11, 5}



return true if possible else false

Dec 124

Q) Minimum Swaps to Make Sequences Increasing.

if $\rightarrow [1, 3, 5, 4] \times$
 $[1, 2, 3, 7] \checkmark$

o/p $\rightarrow [1, 3, 5, 7] \checkmark$
 $[1, 2, 3, 4] \checkmark$

Total swaps required = 1
ans

code: recursion + mem

```
int solve (int n, vector<int> &
nums1, nums2, index, bool
swapped, vector<vector<int>> &
dp) {
```

// base case
if (index == n)
return 0;

if (dp[index][swapped] != -1)
return dp[index][swapped];

int ans = INT_MAX;
int prev1 = nums1[index+1];
int prev2 = nums2[index+1];

```
if (swapped) {
    swap (prev2, prev1);
}
```

// no swap

```
if (nums1[index] > prev1 &&
    nums2[index] > prev2) {
    ans = solve (n, nums1, nums2,
index+1, 1, dp);
```

// swap

```
if (nums1[index] > prev2 &&
    nums2[index] > prev1) {
    ans = min (ans, 1 + solve (n,
nums1, nums2, index+1, 1, dp));
}
```

return dp[index][swapped] = ans;

```
int minSwap (vector<int> & nums1,
vector<int> & nums2) {
    nums1.insert (begin(), 1);
    int n = nums1.size();
    nums2.insert (begin(), 1);
    vector<vector<int>> dp (n,
vector<int> (2, 1));
    return solve (n, nums1, nums2, 1, 0, dp);
}
```

Tabulation

index = 0 $\rightarrow$ n-1
swapped = 0 $\rightarrow$ 1

```
int solve (nums1, nums2) {
    int n = nums1.size();
    vector<vector<int>> dp (n+1,
vector<int> (2, 0));
```

```
for (index = n-1; index >= 1; index--) {
    for (swapped = 0; swapped <= 1;
swapped++) {
```

```
int ans = INT_MAX;
int prev1 = nums1[index+1];
int prev2 = nums2[index+1];
```

```
if (swapped)
    swap (prev1, prev2);
```

```
if (nums1[index] > prev1 &&
    nums2[index] > prev2) {
```

```
ans = dp[index+1][0];
}
```

```
if (nums1[index] > prev2 &&
    nums2[index] > prev1) {
```

ans = min (ans, 1 + dp[index+1][1];

dp[index][swapped] = ans;

return dp[1][0];

LC 125

Q) Longest Arithmetic Subsequence (AP)

if $\rightarrow$ arr = [1, 7, 10, 13, 14, 17]

o/p = 3 (i.e. 7, 10, 13)
= 4 (1, 7, 13, 19) $\checkmark$
longest

Brute force (Recursion)

- 1) Select any two no. and find difference (assume them as last 2 elements)
- 2) Now see how many elements exist with that difference.

recursion + mem (TLE)

```
int solve (int index, int diff,
int arr[], unordered_map<int,
int> & dp) {
```

// backward check

```
if (index < 0)
    return 0;
```


Tabulation

```
if (dp[index][count(diff)])
    return dp[index][diff];
```

```
int ans = 0;
for (int j = index-1; j >= 0; j--) {
    if (A[index] - A[j] == diff) {
        ans = max(ans, 1 +
        solve(j, diff, A, dp));
    }
}
```

```
return dp[index][diff] = ans;
```

```
}
```

```
int length of LAS (int A[], int n) {
    if (n <= 2)
        return n;
```

```
int ans = 0;
```

```
unordered_map <int, int> dp[n];
```

```
for (int i = 0; i < n; i++) {
    for (int j = i+1; j < n; j++) {
```

```
ans = max(ans, 2 +
```

```
solve(i, A[j] - A[i], A,
```

```
dp);
}
```

```
return ans;
```

```
int length of LAS (int A[], int n) {
    if (n <= 2) return n;
    int ans = 0;
    unordered_map <int, int> dp[n];
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            int diff = A[i] - A[j];
            int count = 1;
```

// check if ans is already present.

```
if (dp[j].count(diff))
    count = dp[j][diff];
dp[i][diff] = 1 + count;
ans = max(ans, dp[i][diff]);
}
```

```
return ans;
```

Q.126

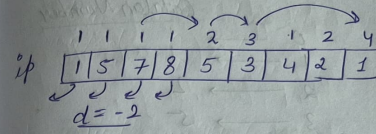
Q.1) Longest AP with given 'd'
(common difference)

e.g. i/p → 1 2 3 5 7 9 d = 2

o/p → 5

(ie 1 3 5 7 9)

approach



go from left to right.
At each element check if a-d
term exist on it's left side of
array.

if yes → add its ans + 1

if no → assign

return max ans i.e → 4

code

```
int LAS (vector <int> arr, int diff) {
    unordered_map <int, int> dp;
    int ans = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
int temp = arr[i] - diff;
```

```
int tempans = 0;
```

```
if (dp.count(temp))
```

```
tempans = dp[temp];
```

```
dp[arr[i]] = 1 + tempans;
```

```
ans = max(ans, dp[arr[i]]);
```

```
return ans;
```

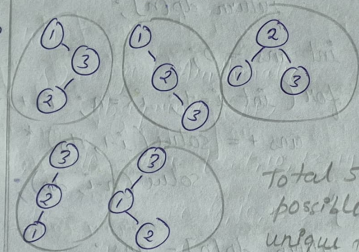
lec 127

Q.1) Unique Binary Search Trees

i/p → int n

e.g = n = 3

o/p → 5

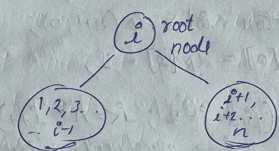


Total 5
possible
unique trees
(BST)
using all 'n' nodes

approach

traverse from left to right and
assume each element as a root
node

1 2 3 4 5 6 ... i ... n



$$f(n) = \sum f(i-1) * f(n-i)$$

code

```
int solve (int n, vector<int> &dp) {  
    if (n <= 1)  
        return 1;  
    if (dp[n] != -1)  
        return dp[n];  
    int ans = 0;  
    for (int i = 1; i <= n; i++) {  
        ans += solve(i-1, dp) *  
            solve(n-i, dp);  
    }  
    return dp[n] = ans;  
}
```

```
int numTrees (int n) {  
    vector<int> dp(n+1, -1);  
    return solve(n, dp);  
}
```

```
int solveTab (int n) {  
    vector<int> dp(n+1, 0);  
    dp[0] = dp[1] = 1;  
    for (int i = 2; i <= n; i++) {  
        // i → number of nodes  
        for (int j = 1; j <= i; j++) {  
            dp[i] += dp[j-1] * dp[i-j];  
        }  
    }  
    return dp[n];  
}
```

n	C _n
0	1
1	1
2	2
3	5
n	$C_n = \frac{1}{n+1} \binom{2n}{n}$

ex 28

Q.) Guess Number Higher or Lower.

A → pick no. from 1 to n.
B → guess a no.
B → if correct, A wins.
else.
A → A & B will tell higher or lower.
if B guess x no. and if it's
wrong → B have to pay
cost 'x'.

given a 'n' → tell minimum
amount that B have to pay
for a guaranteed win.

```
int solve (int start, int end) {  
    if (start >= end) {  
        return 0;  
    }  
    int maxi = INT_MAX;  
    for (int i = start; i <= end; i++) {  
        maxi = min(maxi, 1 + max(  
            solve(start, i-1),  
            solve(i+1, end)));  
    }  
    return maxi;  
}
```

```
int getMoneyAmount (int n) {  
    return solve(1, n);  
}
```

recur + mem

```
int solve (start, end, dp) {  
    if (start >= end) {  
        return 0;  
    }
```

```
    if (dp[start][end] != -1)  
        return dp[start][end];
```

```
    int maxi = INT_MAX;  
    for (int i = start; i <= end; i++) {  
        maxi = min(maxi, 1 + max(  
            solve(start, i-1, dp),  
            solve(i+1, end, dp)));  
    }  
    return dp[start][end] = maxi;
```

ex 28

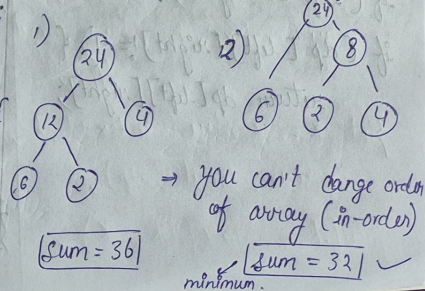
Q.) Minimum cost tree from leaf values.

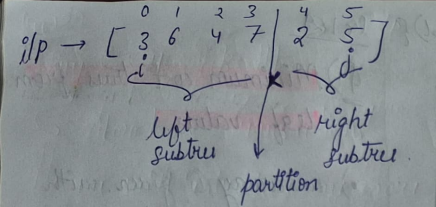
- an array is given, with positive int values.
- The values of arr corresponds to the values of each leaf in an in-order traversal.
- Consider all binary trees
- The values of each non-leaf node is equal to the product of the largest leaf value in its left & right subtree.

→ return smallest possible sum of the values of each non-leaf node.

eg arr = [6, 2, 4]

possible BST → only Binary Tree.





$f(i, j) \rightarrow$ represents sum of all non-leaf nodes for range i to j .

$$f(i, j) = \min \left[\max(i, k) * \max(k+1, j) + f(i, k) + f(k+1, j) \right]$$

↙ ↘
recursive call

$f(0, 0) = 0 \rightarrow$ single leaf node if $(i = j)$.

code rec + mem

```
int solve (vector<int> & arr,
map<pair<int, int>, int> & maxi,
int left, int right, vector<int> & dp) {
    if (left == right) return 0;
    if (dp[left][right] != -1) {
        return dp[left][right];
    }
}
```

int ans = INT_MAX;

for (int i = left; i < right; i++)

ans = min (maxi [i left, i],

maxi [i+1, right] +

solve (arr, maxi, left, i, dp) +

solve (arr, maxi, i+1, right, dp);

return dp [left] [right] = ans;

int mainCost (vector<int> & arr) {

map<pair<int, int>, int> maxi;

for (int i = 0; i < arr.size(); i++)

maxi [i, i] = arr[i];

for (int j = i+1; j < arr.size(); j++)

maxi [i, j] = max (arr[j],

maxi [i, j-1])

}

vector<vector<int>> & dp (

n+1, vector<int> (n+1, -1));

return solve (arr, maxi, 0, arr.size()-1,

dp);

}

lec 130

q. Best Time to Buy & Sell stocks

leetcode 121 $\rightarrow$ solved.

prices = { 7, 1, 5, 3, 6, 4 }

↙ ↘
buy sell

profit = sell - buy = 5

lec 131 leetcode $\rightarrow$ 122

$\hookrightarrow$ you buy & sell multiple times

prices = { 7, 1, 5, 3, 6, 4 }

↙ ↘ ↙ ↘
B S B S

profit = 7

code

```
int solve (int index, int buy,
vector<int> & prices, vector<vector> & dp) {
    if (index >= prices.size())
        return 0;
    if (dp[index][buy] != -1)
        return dp[index][buy];
}
```

int profit = 0;

if (buy) {

int buyKaro = -prices [index]

+ solve (index+1, 0,

prices, dp);

int skipKaro = 0 + solve (index+1,

1, prices, dp);

profit = max (buyKaro, skipKaro);

} else {

int sellKaro = prices [index] +

solve (index+1, 0, prices, dp);

int skipKaro = 0 + solve (index,

1, prices, dp);

profit = max (sellKaro,

skipKaro);

}

return dp [index] [buy] = profit;

int maxProfit (vector<int> & prices) {

int n = prices.size();

vector<vector<int>> dp (n, vector<int> (2, -1));

return solve (0, 1, prices, dp);

}

lec 132

→ 3D dp

Buy & sell stocks III→ Atmost 2 transactions

→ we will introduce one more variable in parameter.

```
int solve (int index, int buy,
            index <int> & prices, int limit,
            vector <vector <vector <int>>> & dp) {
```

```
if (index == prices.size())
    return 0;
```

```
if (limit == 0)
    return 0;
```

```
int profit = 0;
```

```
if (dp[index][buy][limit] != -1)
    return dp[index][buy][limit];
```

```
if (buy) {
    int bk = -prices[index] +
    solve (index+1, 0, prices, limit,
           dp);
```

```
int sk = solve (index+1, 1,
                prices, limit, dp);
```

```
profit = max (bk, sk);
```

else {

```
int sell = prices [index] +
```

```
solve (index+1, 1, prices, limit, dp);
```

```
int skip = 0 + solve (index+1, 0,
                      prices, limit, dp);
```

```
profit = max (sell, skip);
```

}

```
return dp [index] [buy] [limit]
```

```
= profit;
```

}

```
int maxprofit (vector prices) {
```

```
vector <vector <vector <int>>>
```

```
dp (n, vector <vector <int>>
```

```
( 2, vector <int> (3, -1));
```

```
return solve (0, 1, prices, 2,
              dp);
```

}

}

lec 133

Buy & sell stocks IV.→ Atmost 'k' transactions.

soln → Just replace 2 by k in previous soln & update limit = k.

lec 134

Buy & sell stocks with Transaction fee.

int fee → const for all transactions.

Note

→ if transaction fee is greater than profit then avoid that transaction.

soln

after sell transaction subtract fee from profit.

lec 135

* LCS(Q.) Longest Common Subsequence

e.g. a = "a b c d e f"
b = "a c e z"

→ o/p = "ace" ✓

eg

"a b c d e f"
"a c e z"

if (match) {

```
return = 1 + f(a, b, i+1, j);
```

else {

```
return max [f(a, b, i+1, j),
            f(a, b, i, j+1)]
```

recursion + mem

if (i >

```
int solve (string &a, string &b,
            int i, int j, vector <vector <int>>
            & dp) {
```

```
if (i >= a.size()) return 0;
```

```
if (j >= b.size()) return 0;
```

```
if (dp[i][j] != -1) return dp[i][j];
```

```
int ans = 0;
```

```
if (a[i] == b[j]) {
    ans = 1 + solve (a, b, i+1, j+1, dp);
```

} else {

```
ans = max (solve (a, b, i+1, j, dp),
```

```
solve (a, b, i, j+1, dp));
```

}

```
return dp[i][j] = ans;
```